

CSE 719 — Lecture 10: RPCs & Concurrency Control

Past-Paper Solutions (2020–2024)

Exam: Wed 1 Jul 2026 · Yield: 5/5 · $\approx 55\pm$ marks covered

Quick Reference

RPC: Call from one function to another where caller and callee reside in *different processes* (possibly different hosts). Procedure address = IP + port + procedure number. Counterpart: RMI in object-based settings. Proposed by Birrell & Nelson (1984).

Key RPC components:

- **Client stub** — same function signature as callee; marshals arguments; allows same caller code for LPC and RPC
- **Communication module** — forwards requests and replies to appropriate hosts
- **Dispatcher** — selects which server stub to route the request to
- **Server stub** — unmarshals arguments; calls callee; marshals return value

Call semantics:

Retransmit?	Filter dups?	Re-execute or Retransmit reply?	Semantics	Example
Yes	No	Re-execute	At least once	Sun RPC
Yes	Yes	Retransmit reply	At most once	Java RMI
No	N/A	N/A	Maybe	CORBA

ACID: Atomicity (all-or-nothing) · Consistency (server stays consistent) · Isolation (indivisible from other transactions) · Durability (committed effects survive crashes).

Concurrency problems: Lost Update (two txns overwrite each other) · Inconsistent Retrieval (txn reads partially updated data).

Serial equivalence: Interleaving is serially equivalent iff all pairs of conflicting operations from different transactions execute in the same order for every shared object. Conflict = same object, at least one write.

Deadlock: Circular wait. Detected via Wait-For-Graph (WFG). Prevented via 2PL or timestamp ordering.

RPC: Definition, Motivation, Design Issues

[2021 Q8a (2.75 mk), 2022 Q8a (3 mk), 2020 Q3a (1.5 mk)]

What is RPC?

A **Remote Procedure Call (RPC)** is a mechanism that allows a function in one process (caller) to invoke a function residing in a *different process* — possibly on a different machine — in the same way as a local procedure call, without the programmer explicitly coding the network communication.

- Caller and callee are in different processes; the function call crosses a process boundary.
- Cannot use pointers across processes (address in P1 is meaningless in P2); instead uses *global references*: IP address + port + procedure number.
- Object-based counterpart: **Remote Method Invocation (RMI)**.
- Code for client stub, dispatcher, server stub is *generated automatically* from function signatures by middleware (CORBA, Sun RPC, Java RMI).
- Enables code reuse, transparency, and modularity in distributed systems.

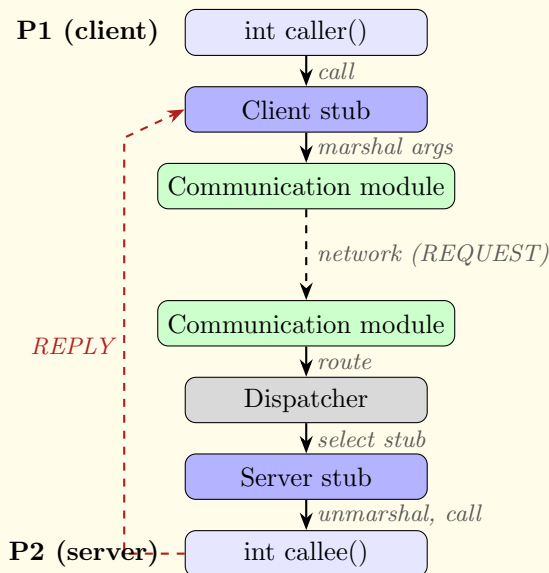
Primary motivation: Allow processes to call functions in other processes with the same familiar call syntax as local calls, hiding all network complexity in the middleware layer. This enables distributed system construction through modular procedure invocation.

Design Issues for RPC:

1. **Call semantics** — Cannot guarantee exactly-once under failures. Must choose: at-most-once (filter duplicates, retransmit reply), at-least-once (retransmit + re-execute), or maybe (no retransmit).
2. **Parameter passing / Marshalling** — Caller and callee may use different data representations (big-endian vs little-endian). Middleware converts arguments to *Common Data Representation (CDR)* before transmission (marshalling) and back on receipt (unmarshalling).
3. **Binding** — How does the client locate the server? Static binding (hardcoded) or dynamic binding (name server lookup at runtime).
4. **Transport protocol** — UDP (lightweight, needs RRA retransmit logic) vs TCP (reliable but heavier). Choice affects latency and failure handling.
5. **Fault tolerance** — Server may crash before or after executing the call; client cannot distinguish these cases. Need idempotent operations or unique request IDs.
6. **Security** — Authentication (is the server who it claims to be?) and authorization. RPCs cross trust boundaries.
7. **Heterogeneity** — Caller and callee may run on different OSes, languages, hardware. CDR and stub generation solve this.
8. **Performance** — RPC is much slower than LPC (network round trip, marshalling overhead). Must minimise unnecessary RPCs (e.g., AFS whole-file caching).

Schematic Diagram of RPC Mechanism

[2020 Q3a (3 mk)]



- **Client stub:** has the same signature as `callee()`; allows `caller()` code to be identical whether making LPC or RPC.
- **Communication module:** forwards marshalled request to server host; receives reply, returns to stub.
- **Dispatcher:** on server, selects correct server stub based on procedure ID in the request message.
- **Server stub:** unpacks (unmarshals) arguments; invokes actual `callee()`; marshals return value into `REPLY`.

Stub: What Is It, How Generated, and Its Purpose

[2020 Q3b (4.25 mk)]

What is a stub?

A **stub** is a piece of middleware code that acts as a local proxy for a remote function. It has the *same function signature* as the remote callee so that the caller can invoke it as if calling a local function — hiding all network communication details.

There are two stubs:

- **Client stub:** Lives in the caller's process. Marshals arguments into CDR format, sends the REQUEST message, waits for REPLY, unmarshals the result, and returns it to the caller.
- **Server stub:** Lives in the callee's process. Receives the REQUEST from the dispatcher, unmarshals arguments, calls the actual `callee()` function, marshals the return value, sends the REPLY.

How are stubs generated?

Stubs are **generated automatically** by a middleware compiler — the programmer does *not* write them by hand:

1. Programmer defines the function interface in an **Interface Definition Language (IDL)** (e.g., Sun XDR for Sun RPC, CORBA IDL for CORBA).
2. The IDL file is fed into a **stub compiler** (e.g., `rpcgen` for Sun RPC).
3. The compiler outputs: client stub code, server stub code, and marshalling/unmarshalling routines for all parameter types.
4. Programmer only writes: the `caller()` business logic and the `callee()` implementation.

Functionality and purpose of stubs:

- **Transparency:** The caller does not know whether it is invoking a local or remote function.
- **Marshalling:** Converts arguments from platform-dependent format to CDR; handles endianness, data type sizes, pointer elimination.
- **Call semantics enforcement:** Implements retransmit logic, duplicate detection, and reply caching to achieve at-most-once or at-least-once semantics.
- **Decoupling:** The programmer focuses on business logic; the stub handles all distributed systems complexity.

RRA Protocol and Concurrent Extension

[2020 Q5a (RRA steps + figure), 2020 Q5d (2.75 mk) (concurrent access to multiple servers)]

Communication Protocol in RPC: the RRA Protocol

The **Request-Reply-Acknowledge (RRA)** protocol is used by RPC over UDP to achieve reliable, at-most-once communication without TCP's overhead.

Steps of RRA:

1. **Client sends REQUEST:** message contains $\langle request_id, procedure_id, arguments \rangle$. The *request_id* uniquely identifies this invocation.
2. **Server executes** the procedure. Server stores the result (reply) indexed by *request_id*.
3. **Server sends REPLY:** message contains $\langle request_id, result \rangle$.
4. **Client receives REPLY**, extracts result, returns to caller.
5. **Client sends ACKNOWLEDGE:** message contains $\langle request_id \rangle$, telling the server the reply was received.

6. **Server discards stored reply** upon receiving ACK.

RRA figure:

Client		Server
send REQUEST(req_id, proc, args)	→	receive, execute store reply
wait	←	send REPLY(req_id, result)
receive result		wait for ACK
send ACKNOWLEDGE(req_id)	→	discard stored reply

Why store the reply? If the REPLY message is lost, the client retransmits REQUEST. The server detects the duplicate via *request_id* and resends the *cached* reply without re-executing the procedure. This achieves **at-most-once** semantics (filter duplicates + retransmit reply).

Extended RRA for Concurrent Access to Multiple Servers (Q5d):

When a client transaction involves multiple servers (e.g., booking a flight *and* a hotel), sequential RRA wastes time. The protocol is redefined:

1. **Client sends REQUEST simultaneously to all k servers:** each server gets its own request with the same transaction-level request ID but a server-specific procedure ID.
2. **All servers execute concurrently:** servers process their portions in parallel.
3. **Client waits for all k REPLY messages:** collects results from each server.
4. **Client sends ACKNOWLEDGE to all k servers:** each server discards its stored reply.

Client		Server ₁ , Server ₂ , ..., Server _k
send REQUEST to all k	→	all execute in parallel
wait for k REPLYs	←	all send REPLY
send ACK to all k	→	all discard replies

Risk: If any server fails after receiving the REQUEST but before sending a REPLY, the client waits indefinitely (or until timeout). Requires 2PC or abort protocol to handle partial failures.

Callback RPC and Lightweight RPC

[2020 Q5c (2 mk)]

Callback RPC:

In standard RPC, the client always initiates calls to the server. **Callback RPC** reverses this: the *server initiates an RPC back to the client* when an asynchronous event occurs.

- The client registers a *callback function* with the server (via an initial RPC).
- The server stores the client's address and callback procedure ID.
- When the event occurs, the server calls the client's callback function using RPC.
- **Use case:** Asynchronous notifications — e.g., AFS sends a *callback break* to Venus when a cached file is modified. The server “calls back” to notify the client without the client polling.

Lightweight RPC (LRPC):

Standard RPC has significant overhead even when caller and callee are on the *same machine* (marshalling, kernel crossings, network stack traversal). **Lightweight RPC** is optimised for intra-machine cross-process calls:

- Uses **shared memory** (A-stack: argument stack) instead of network messages to pass arguments. No marshalling/unmarshalling needed.
- **S-stub** (server stub) is called directly via a trap to the kernel; no communication module

traversal.

- Eliminates network stack, reduces context switches, and avoids copying.
- **Result:** 2–10× faster than standard RPC for same-machine calls.
- **Use case:** Microkernel architectures where OS services (file system, memory manager) run as separate processes on the same machine.

Marshalling, Unmarshalling, and Communication Paradigms

[2021 Q1d (2 mk) (partial), 2024 Q5c (4 mk)]

Marshalling and Unmarshalling in RPC:

Different hardware architectures store data differently:

- **Big-endian** (IBM z, System 360): most significant byte at lowest address.
- **Little-endian** (Intel x86): least significant byte at lowest address.

RPC middleware uses a **Common Data Representation (CDR)** — a platform-independent wire format.

During invocation:

1. **Marshalling (at client stub):** Caller's arguments are converted from platform-dependent format → CDR. Pointers are replaced with global references (IP + port + ID). Packed into the REQUEST message.
2. **Transmission:** CDR-encoded message sent over network.
3. **Unmarshalling (at server stub):** CDR arguments extracted and converted to server's platform-dependent format. Passed to `callee()`.
4. **Return:** Return value marshalled at server stub (CDR) → sent in REPLY → unmarshalled at client stub → returned to caller.

Evaluation of Communication Paradigms for Time-Sensitive Applications:

Paradigm	Performance	Reliability
Message Passing	Low overhead per message; no marshalling if same platform; non-blocking async send possible; high throughput for bulk data	Must implement reliability manually; no built-in duplicate detection; failure handling is programmer's responsibility
RPC / RMI	Higher overhead: marshalling, CDR conversion, two kernel crossings, round-trip wait; synchronous (blocks caller); poor for streaming	Built-in call semantics (at-most-once/at-least-once); automatic retransmit; duplicate filtering; stub handles failures transparently

For time-sensitive applications (e.g., video streaming, real-time trading):

- **Message passing preferred for throughput:** async, non-blocking sends allow the sender to continue without waiting; lower per-message latency; suitable when occasional loss is tolerable (UDP streaming).
- **RPC preferred for correctness:** when each call must produce a verified result (financial transaction, authentication), RPC's at-most-once semantics and synchronous confirmation are essential.
- **Hybrid:** many real-time systems use async message passing for data streams and RPC for control operations (e.g., gRPC with streaming).

ACID Properties

[2021 Q1d (2 mk) (partial), 2024 Q8a (3 mk)]

ACID defines the four correctness properties a transaction must satisfy:

Property	Definition
Atomicity	All-or-nothing principle: a transaction either completes successfully (all its effects are recorded in server objects) or has <i>no effect at all</i> (full rollback on failure). No partial execution is visible.
Consistency	If the server starts in a consistent state, the transaction ends the server in a consistent state. The transaction preserves all integrity constraints (e.g., account balances cannot go negative).
Isolation	Each transaction must be performed without interference from other transactions. Non-final (intermediate) effects of a transaction must not be visible to other transactions until commit. Transactions appear to execute serially even if concurrent.
Durability	After a transaction has completed successfully, all its effects are saved permanently in stable storage and survive subsequent failures (crashes, power loss).

Why these properties are hard in distributed systems:

- **Clients and servers may crash** during a transaction → atomicity requires rollback or 2PC.
- **Multiple concurrent transactions** → isolation requires concurrency control (locking, timestamps).
- **Distributed transactions span multiple servers** → durability requires coordinated write-ahead logging across servers.

Note: ACID under crash scenarios (disk-per-txn, memory+disk/50-txn, log file) is covered in Replication_2PC_Solutions.pdf.

Schedule Trace Analysis (Serial Equivalence)

[2022 Q7a (4 mk)]

Given schedule [$r[X]$ = read object X, $w[X]$ = write object X]:

Transaction T_0	Transaction T_1
$r[X]$ $w[X]$	$r[X]$ $r[Y]$ commit
$r[Y]$ $w[Y]$ commit	

Step 1 — Identify conflicting operation pairs (same object, at least one write, one op from each transaction):

Object X:

- $T_0:w[X]$ (step 2) vs $T_1:r[X]$ (step 3) $\rightarrow$ *write-read conflict*. T_0 executed first $\rightarrow$ mark (T_0 , T_1).

Object Y:

- $T_1:r[Y]$ (step 4) vs $T_0:w[Y]$ (step 7) $\rightarrow$ *read-write conflict*. T_1 executed first $\rightarrow$ mark (T_1 , T_0).

Step 2 — Check consistency:

- For object X: T_0 precedes $T_1 \Rightarrow (T_0, T_1)$.
- For object Y: T_1 precedes $T_0 \Rightarrow (T_1, T_0)$.
- The two pairs are **inconsistent** (mixed ordering).

Conclusion: NOT serially equivalent. This transaction schedule will *not* go off without a hitch.

Issue raised: Inconsistent Retrieval Problem.

T_1 reads X *after* T_0 has already written the new value of X (gets T_0 's updated X), but reads Y *before* T_0 updates Y (gets the old value of Y). T_1 therefore sees an inconsistent snapshot: part of T_0 's state change (X updated) but not the rest (Y not yet updated). If X and Y are related (e.g., seat counts on two legs of a journey), T_1 's computation will be incorrect.

Fix: Use locking (2PL) or timestamp ordering to prevent T_1 from reading Y until T_0 commits.

Concurrency Control + Deadlock + Locking Schemes

[2021 Q6a (2.75 mk), 2022 Q6a (3 mk), 2022 Q7b (3 mk)]

Concurrency Control is the set of mechanisms that ensures transactions execute concurrently while maintaining ACID (especially Isolation). Its goal: *increase throughput (transactions per second) without violating correctness*.

Why needed: Without concurrency control, concurrent transactions produce Lost Update and Inconsistent Retrieval problems (demonstrated by serial-equivalence violations above).

Approaches to preventing isolation violations (Q7b):

1. Pessimistic concurrency control — Locking:

- *Assume conflicts are likely*; prevent two transactions from accessing the same object simultaneously.
- **Exclusive locking:** Each object has one lock. Transaction T calls `lock(O)` before any read or write on O; blocks if another transaction is inside. Calls `unlock(O)` when done. Guarantees isolation but reduces concurrency (read-read is safe, yet exclusive locking blocks it).
- **Read-write locks:** Lock has two modes.
 - **Read mode:** Multiple transactions allowed concurrently (read-read is not a conflict).
 - **Write mode:** Exclusive; blocks all other readers and writers.
 - `read_lock(O)`: granted if no transaction holds write-mode lock on O.
 - `write_lock(O)`: granted only if no other transaction holds any lock on O.
 - **Lock promotion:** If T holds `read_lock(O)` and wants to write, calls `write_lock(O)`; succeeds only if T is the only reader (no other readers or writers).
- **Two-Phase Locking (2PL):** Guarantees serial equivalence with read-write locks.
 - *Growing phase:* T can only acquire or promote locks; cannot release any.
 - *Shrinking phase:* T can only release locks; cannot acquire or promote.

- A transaction cannot acquire a lock after it has released any lock.
- *Strict 2PL*: All locks released only at commit/abort point. Prevents dirty reads.

2. Optimistic concurrency control — Check at commit:

- Assume conflicts are rare; allow transactions to execute freely, check for serial equivalence at commit time.
- At commit: check if the transaction's reads and writes conflict with any concurrent transaction.
- If not serially equivalent $\rightarrow$ **abort** and roll back all writes; retry.
- Avoids lock overhead during execution; wastes work if conflicts are frequent.
- **MVCC (Multi-Version CC)**: Each write creates a new version; readers access the version current at their start timestamp; writers proceed without blocking readers.

Deadlock in concurrency control:

A **deadlock** occurs when two or more transactions are *each waiting for a lock held by the other*, forming a circular wait from which no transaction can proceed.

Classic example:

- T_1 holds lock on object A, waits for lock on B.
- T_2 holds lock on object B, waits for lock on A.
- Neither can proceed $\rightarrow$ deadlock.

Necessary conditions for deadlock (all four must hold):

1. **Mutual exclusion**: At least one resource held in a non-shareable mode.
2. **Hold and wait**: A transaction holds a resource while waiting for another.
3. **No preemption**: Locks cannot be forcibly taken from a transaction.
4. **Circular wait**: $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_1$ cycle in the wait-for graph.

Locking schemes and deadlock: Two-phase locking *prevents* serial-equivalence violations but *does not prevent deadlocks* — 2PL can still produce circular waits. Deadlock must be detected and broken separately.

Timestamp Ordering

[2021 Q6b (2 mk)]

Timestamp ordering is an optimistic concurrency control mechanism that assigns each transaction a unique timestamp when it opens, and enforces that all conflicting accesses occur in timestamp order.

Rule: For each server object, record:

- **max_read_ts**: the maximum timestamp of any transaction that has read this object.
- **max_write_ts**: the maximum timestamp of any transaction that has written this object.

When transaction T (timestamp ts_T) accesses object O :

- **T wants to write O**: If $ts_T < \text{max_read_ts}(O)$ or $ts_T < \text{max_write_ts}(O) \rightarrow$ **abort T** (a later transaction has already used the old value; T's write is "too late").
- **T wants to read O**: If $ts_T < \text{max_write_ts}(O) \rightarrow$ **abort T** (a later transaction has already overwritten the value T expects to read).

Two phases of timestamp ordering:

1. **Validation phase**: Before T commits, check whether all reads and writes by T are consistent

- with the timestamp ordering rules for all objects accessed. If any conflict detected $\rightarrow$ abort T.
2. **Update phase:** If validation passes, apply all of T's *tentative writes* permanently to the server objects and update `max_write_ts` for each written object.

Advantage over locking: No deadlocks (no waiting; conflicts cause abort instead). Good when conflicts are rare. **Disadvantage:** High abort rate under heavy write contention; repeated retries waste work.

Deadlock Detection + Wait-For-Graph (WFG)

[2022 Q7c (2 mk), 2024 Q3c (3 mk)]

Wait-For-Graph (WFG): Purpose

A **Wait-For-Graph (WFG)** is a directed graph used to detect deadlocks in a system using locking:

- **Nodes:** one per active transaction.
- **Edge $T_i \rightarrow T_j$:** transaction T_i is waiting for T_j to release a lock it holds.
- **Deadlock $\Leftrightarrow$ cycle in WFG.** A cycle means a set of transactions each waiting for the next in the cycle $\rightarrow$ none can proceed.

Example WFG:

T_1 holds lock on A, waits for B (held by T_2) $\Rightarrow$ edge $T_1 \rightarrow T_2$
 T_2 holds lock on B, waits for C (held by T_3) $\Rightarrow$ edge $T_2 \rightarrow T_3$
 T_3 holds lock on C, waits for A (held by T_1) $\Rightarrow$ edge $T_3 \rightarrow T_1$

Cycle: $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1 \Rightarrow$ **deadlock detected**. Resolution: abort one transaction (e.g., T_3) to break the cycle; others proceed.

When do deadlocks occur? When all four necessary conditions hold simultaneously: mutual exclusion, hold-and-wait, no preemption, and circular wait (cycle in WFG).

Deadlock Detection in Distributed Systems (Q7c, Q3c):

In a distributed system, transactions hold locks on objects across *multiple servers*. No single server sees the full WFG $\rightarrow$ local WFG at each server is incomplete.

Approach — Global WFG construction:

1. Each server maintains a *local* WFG of transactions waiting for locks on its objects.
2. A **deadlock detector** (centralised or distributed) periodically collects local WFGs from all servers.
3. Merges them into a *global WFG*.
4. Searches for cycles in the global WFG $\rightarrow$ deadlock found.
5. Picks a *victim* transaction (e.g., youngest or fewest locks) and aborts it; informs all servers holding locks for that transaction to release them.

Conditions a deadlock detection algorithm must satisfy:

1. **Safety (no false positives):** The algorithm must *only* report a deadlock when a real deadlock actually exists. Reporting a false deadlock would cause unnecessary transaction aborts.
2. **Liveness (no false negatives):** The algorithm must *eventually* detect every real deadlock that exists. Failing to detect a deadlock leaves transactions stuck forever.

Challenge in distributed systems: Network delays mean a lock may be released (deadlock broken) before the global WFG snapshot is complete. This can cause *phantom deadlocks* (false

positives) — the algorithm detects a cycle that no longer exists. Careful timestamp-based snapshot protocols are required.

Distributed vs Centralized Concurrency

[2024 Q6a (3 mk)]

How concurrency in distributed systems differs fundamentally from centralized systems:

1. **No global clock:** In a centralized system, the OS provides a single monotonic clock; timestamps and total ordering of events are trivial. In a distributed system, clocks on different nodes drift independently and cannot be perfectly synchronized. There is no global “now.” Transactions on different servers have no universally agreed ordering without coordination (e.g., logical clocks, vector clocks). This makes timestamp ordering and serialization far harder.
2. **Partial failure:** A centralized server either runs or crashes — the failure is total. In a distributed transaction spanning multiple servers, *some* servers may commit while others crash → partial commit. This violates atomicity. Distributed protocols (2PC) must coordinate to ensure all-or-nothing commit, but 2PC itself is blocked if the coordinator crashes.
3. **No global lock manager:** A centralized system has one lock table; acquiring all necessary locks is straightforward. In a distributed system, locks for a single transaction may be held on multiple servers. No single server knows the complete WFG → deadlock detection requires global coordination. Lock acquisition across servers introduces extra message round trips and latency.
4. **Network delays introduce uncertainty:** A server cannot distinguish “client is slow” from “client has crashed.” Timeouts are imprecise. A transaction may be retried even though it already executed (RPC exactly-once problem), risking double-commit or double-abort.
5. **Concurrent access is structural, not exceptional:** In a centralized system, concurrency control is an optimization concern. In a distributed system, *every* client request is concurrent with others by nature (no shared address space, no single scheduler). Isolation must be enforced through explicit message-passing protocols rather than kernel synchronization primitives.

Impact on design: Distributed concurrency control must tolerate message loss, process failures, and clock skew — making it orders of magnitude more complex than centralized equivalents.

Deadlock Scenario: Rowing Crew (Boittha Problem)

[2024 Q8b (3 mk)]

Scenario: Hasan, Suman, and Aman (3 rowers). One boat, 4 boitthas (oars). Each rower needs *exactly 2* boitthas to row. Protocol every rower follows:

```
while (!at_destination) {
    recover_strength();
    grab_one_boittha(); // acquire resource #1
    grab_one_boittha(); // acquire resource #2
    row_the_boat(); row_the_boat();
    drop_one_boittha(); drop_one_boittha();
}
```

Question: Is it possible for this plan to lead to deadlock?

Answer: No, deadlock is not possible with this setup.

Justification:

Deadlock requires a circular wait in which *every* thread is blocked waiting for a resource held by

another thread in the cycle, and *no* resource can be freed. With 3 rowers and 4 boitthas:

1. Consider the worst-case simultaneous grab scenario: all three rowers each grab their *first* boittha simultaneously. After 3 grabs, 1 boittha ($= 4 - 3$) remains available.
2. The remaining (4th) boittha can be grabbed by *at least one* rower as their second boittha — that rower now has 2 boitthas and can row.
3. A circular wait requires *all* threads to be blocked. But the rower who grabs the 4th boittha is *not* blocked — it proceeds to row, then drops 2 boitthas back. This frees 2 boitthas for the other two rowers.
4. No cycle can form in the WFG: there is always at least one rower who can make progress.

Key principle: This is analogous to the Dining Philosophers problem with N philosophers and $N + 1$ forks. With N philosophers and $N + 1$ resources (each needing 2), deadlock is *impossible* because there are always enough resources for at least one philosopher to complete. Here $N = 3$ rowers, $N + 1 = 4$ boitthas.

If there were only 3 boitthas (not 4), deadlock *would* be possible: all three rowers grab one boittha each, leaving 0 remaining, and all three wait for their second boittha indefinitely $\rightarrow$ circular wait $\rightarrow$ deadlock.

Summary Table

Year / Q	Topic	Marks
2020 Q3a	RPC motivation + schematic diagram	4.5
2020 Q3b	Stub: what, how generated, purpose	4.25
2020 Q5a	RRA protocol steps + figure	~ 3
2020 Q5c	Callback RPC + Lightweight RPC	2
2020 Q5d	Concurrent RRA for multiple servers	2.75
2021 Q1d	Marshalling/Unmarshalling + ACID	2
2021 Q6a	Concurrency control + deadlock + locking	2.75
2021 Q6b	Timestamp ordering (validation + update phases)	2
2021 Q8a	What is RPC + design issues	2.75
2022 Q6a	Deadlock + locking schemes in concurrency control	3
2022 Q7a	Schedule trace: T_0/T_1 serial-equivalence analysis	4
2022 Q7b	Concurrency control + preventing isolation violation	3
2022 Q7c	Deadlock detection in distributed systems; when occur	2
2022 Q8a	What is RPC + design issues (same as 2021)	3
2024 Q3c	WFG purpose + example + algorithm conditions	3
2024 Q5c	Marshalling + communication paradigm comparison	4
2024 Q6a	Distributed vs centralized concurrency	3
2024 Q8a	ACID definition (one sentence per letter)	3
2024 Q8b	Rowing crew deadlock analysis	3
Total		≈ 57.75 mk

Already covered in other solution PDFs (do NOT repeat):

- 2022 Q3a/b/c (ACID under crashes: disk-per-txn, memory+disk, log file) $\rightarrow$ Replication_2PC_Solutions.pdf
- 2024 Q2a (consensus algorithms + logical vs physical concurrency) $\rightarrow$ Paxos_Solutions.pdf
- 2024 Q2c (atomic commit + 2PC for nested transactions) $\rightarrow$ Replication_2PC_Solutions.pdf

Star items for final revision:

- **RPC design issues** asked verbatim 2021 & 2022 — know all 8 issues cold.
- **Serial equivalence** check: mark (T_i, T_j) per object; all must match — if mixed $\rightarrow$ not SE.
- **WFG conditions**: safety (no false positives) + liveness (no false negatives).
- **2PL**: growing phase (acquire/promote only), shrinking phase (release only), strict 2PL (release at commit only).
- **Boittha scenario**: N rowers + $N+1$ boitthas $\Rightarrow$ no deadlock; only N rowers $\Rightarrow$ deadlock possible.